

**Auckland University of
Technology**

**Data Process and Software
Modelling**

Progress report

Name: Suemon Kwok

Student ID: 14883335

Table of Contents

Task 1: Organisational and System Analysis.....	3
1.1 Organisation Overview	3
1.2 Problem statement	3
1.3 Departmental Structure and Responsibilities	4
1.4 Stakeholders and User Roles	4
1.5 System Scope, Boundaries, and Assumptions	5
Task 2: Requirements Engineering.....	6
2.1.0 Business requirement / goal / elicitation	6
2.1.0 Requirements Elicitation Methodology	7
2.1.1 Client questions	7
2.2 Functional Requirements	11
2.2.1 Sales Capture and Transaction Tracking	11
2.2.2 Player Demographics and Segmentation	11
2.2.3 Reporting, Analytics, and Export.....	11
2.2.4 Role-Based Access and Security	12
2.3 Non-Functional Requirements	12
2.4 Stakeholder Traceability Matrix	12
Below is a table linking the stakeholders to requirements	12
Task 3: UML Requirements Modelling.....	13
3.1 Use Case Diagram	13
3.2 Fully Dressed Use Case Descriptions: Use cases explained	14
Task 4: Initial Structural Design and Implementation Planning.....	32
4.1 UML Class Diagram.....	32
.....	32
4.2 Class Diagram : How it works	33
4.3 UML to C# Class Mapping.....	33
4.4 OOP Principles Applied	34
4.4.1 Abstraction.....	34
4.4.2 Encapsulation.....	34
4.4.3 Inheritance	34
4.4.4 Polymorphism.....	34
4.5 Anticipated Design Weaknesses and Critical Evaluation	34
4.6 C# Implementation Overview.....	35
4.6 .1 Microtransaction Analytics System demo	36
4.7 GitHub Repository and Compilation Instructions	40
References	41

Data Process and Software Modelling Assignment 1

Project Phase 1 Progress Report

Organisation: Grinding Gear Games | System: Microtransaction Analytics System (MAS)

Task 1: Organisational and System Analysis

1.1 Organisation Overview

Grinding Gear Games (GGG) is a New Zealand based independent video game developer and publisher founded in Auckland in 2006 according to (www.companyoffice.govt.nz, 2006). The company is primarily known for developing and operating Path of Exile, a free to play action role-playing game that has accumulated tens of millions of registered players worldwide and has been recognised as one of the most commercially successful games in its genre. GGG employs more than 200 full time staff according to (www.pitchbook.com, 2025) and (www.linkedin.com, 2026) across departments including Game Development, Marketing, Data Analytics, Finance and Revenue, and Customer Support according to www.zoominfo.com, and operates from its primary office in Auckland with a secondary presence in the United States.

GGG's revenue model is built entirely around voluntary microtransactions. Players are not required to purchase gameplay advantages; instead, they are offered a broad catalogue of purely cosmetic items: weapon skins, character armour sets, pets, portal effects, hideout decorations, and stash tab expansions - purchasable individually or as part of curated discounted bundles. This monetisation model has generated consistent multi-million-dollar annual revenues tied closely to the release cycle of major content updates, known as leagues, which occur quarterly and each produce a significant spike in player engagement and purchasing activity (Grinding Gear Games, 2024).

GGG has indicated that current database system for tracking cosmetic sales is aging and wish to use a modern and more streamline database tracking system for their highest cosmetic sales.

1.2 Problem statement

Despite generating substantial and sustained revenue through its cosmetic catalogue, Grinding Gear Games currently lacks a centralised, structured analytics system for tracking and interpreting microtransaction data at organisational scale. Purchase records are distributed across transactional databases and game server logs without a unified reporting or aggregation layer. As a consequence, business units such as Marketing, Finance, and Game Development are compelled to rely upon manual data extraction, ad hoc spreadsheet compilation, and delayed reporting cycles to address fundamental commercial questions: which cosmetic items achieve the highest sales volume, which character classes correlate with the greatest player expenditure, which geographic regions represent the largest underserved markets, and which items are consistently underperforming relative to their development investment.

This fragmented operational approach introduces measurable delays into strategic decision-making, elevates the risk of resource misallocation, and substantially limits the organisation's capacity to respond in a timely and informed manner to evolving player spending patterns. This limitation is particularly consequential during league launches: recurring major content releases that occur on a quarterly basis and each generate a significant spike in transactional volume wherein the absence of real time or near real time analytics prevents teams from making agile adjustments to promotional strategies and content prioritisation.

1.3 Departmental Structure and Responsibilities

Grinding Gear Games (GGG) departments are as shown in the table below along with their respective responsibility. This table shows the relevance pertaining to this project.

Department	Primary Responsibility	Relevance to MAS
Marketing	Promotional campaign design; cosmetic catalogue positioning; league-aligned advertising	Consumes regional and demographic sales data to target promotions
Game Development	Creation and release of new cosmetic items; art and design resource allocation across character classes	Uses sales rankings and underperformance flags to guide content pipeline decisions
Finance and Revenue	Revenue forecasting, budget planning, commercial performance monitoring across league cycles	Requires consolidated revenue trend data and bundle vs. individual purchase splits
Data Analytics	Deriving actionable insight from transactional and player behavioural data	Primary power user of the MAS; responsible for data pipeline and quality
Customer Support	Player issue resolution; account management; refund handling	Indirect stakeholder - benefits from anonymised player data governance

1.4 Stakeholders and User Roles

Grinding Gear Games (GGG) stakeholders as shown below in table format. The primary user of the system is the data analyst which have full system access. Other users have restrictions on what data is available for their given role in the company. An external stakeholder is the player which generates microtransaction through their purchases

Stakeholder	Department	System Role / Interest	Type
Data Analyst	Data Analytics	Full system access; responsible for data quality, pipeline, and analytical reporting	Internal
Marketing Manager	Marketing	Views demographic and regional reports; exports data for campaigns; configures promotions	Internal
Finance Officer	Finance/Revenue	Accesses revenue trend and bundle split reports; exports for forecasting and audit	Internal
Game Developer	Game Development	Views item category rankings and underperformance alerts to prioritise content development	Internal
Player	External	Purchase behaviour, demographic data, and spending patterns are the primary subject of the system	External (indirect)

1.5 System Scope, Boundaries, and Assumptions

In Scope

The MAS encompasses: automated ingestion and storage of every microtransaction event; classification of purchases by item type, character class, and player demographics; generation of filterable and exportable analytics reports; revenue trend visualisation across configurable time windows; automated flagging of underperforming items; role-based access control for four distinct internal user roles.

Out of Scope

The system does not replace or modify GGG's existing payment processing infrastructure. It does not manage game content creation, patch deployment, or character balance changes. Customer support ticketing, refund workflows, and player account management are explicitly excluded.

Assumptions

- Transaction data from the existing game server infrastructure is accessible to the MAS via a data integration layer.
- Player accounts are identified by opaque account identifiers; no personally identifiable information such as real names or billing addresses is exposed to the analytics system.
- The system operates as an internal tool on GGG's private network; public-facing access is not required.
- The four defined user roles represent the complete set of internal stakeholders requiring system access in Phase 1.

Task 2: Requirements Engineering

2.1.0 Business requirement / goal / elicitation

The following business requirements are derived from GGG's stated business goal: to identify the highest-selling MTX items in order to allocate development resources effectively, and to identify under-performing categories to diversify the skin catalogue.

BR ID	Business Requirement
BR-01	The organisation shall identify the highest-selling MTX skin category (weapon, armour, pet, portal effect) to determine where design resources should be concentrated.
BR-02	The organisation shall determine the most purchased skin per character class so that class-specific content pipelines can be prioritised in future development cycles.
BR-03	The organisation shall identify the single highest-revenue MTX item across all categories in any selectable reporting period.
BR-04	The organisation shall identify MTX items with below-threshold sales performance so that design effort can be redirected to under-served skin categories and character classes.
BR-05	The organisation shall track which character classes have the fewest purchased skins relative to their active player population, enabling targeted new-skin campaigns.
BR-06	The organisation shall segment purchase data by player region and spending tier to inform region-specific marketing promotions.
BR-07	The organisation shall monitor revenue trends over daily, weekly, and monthly windows so that Finance can forecast MTX income and set budget targets.
BR-08	The organisation shall distinguish bundle purchases from individual item purchases to assess the commercial effectiveness of bundled offers.
BR-09	The organisation shall provide filterable sales reports accessible to Marketing, Finance, and Analytics teams without requiring IT intervention.
BR-10	The organisation shall ensure all player personal data collected for analytics purposes complies with applicable privacy legislation.

2.1.0 Requirements Elicitation Methodology

Requirements were derived through a structured elicitation process comprising three activities: analysis of the client's initial problem statement and business goal; department level discovery interviews structured around 40 targeted questions directed at Marketing, Finance, Game Development, and Data Analytics personnel; and review of the existing database programs in GGG. This multi-source approach ensured requirements captured both operational workflows and strategic business intent (Sommerville, 2016).

2.1.1 Client questions

1. Marketing Department

Questions directed at Marketing Managers and Campaign Analysts responsible for cosmetic catalogue promotion and league-aligned advertising strategy.

Q#	Question (User Perspective)	Requirement Outcome Targeted
Q1	When you need to know which, cosmetic items are selling well, what do you currently have to do to get that information, and how long does it take you?	Establishes current-state pain (FR01, FR04); surfaces need for automated reporting
Q2	Have you ever launched a promotion for a skin or bundle and had no reliable way to know how it performed while it was running?	Confirms need for near-real-time reporting (FR09, FR10, NFR01)
Q3	When you are planning a campaign tied to a league launch, what data do you wish you had about player spending behaviour ahead of time?	Surfaces demographic and spending-tier segmentation needs (FR06, FR07, FR09)
Q4	Are there times where you have had to guess which character classes or regions to target for a promotion because the data just was not available?	Confirms need for class- and region-based filtering (FR03, FR06, FR09)
Q5	How do you currently decide when to retire or discount a cosmetic item that is not selling, and what information drives that decision?	Confirms need for underperformance flagging (FR11)
Q6	When you want to compare bundle sales against individual item sales, is that something you can do today, and what does that process look like?	Confirms bundle vs individual tracking requirement (FR05, FR08)
Q7	If you needed to export a filtered report of MTX sales by region and item type to share with a senior stakeholder, how would you do that right now?	Confirms export capability requirement (FR12, FR14)
Q8	Do you have any sense of which types of players, by spending level or account age, respond most strongly to promotional discounts?	Surfaces spending tier segmentation need (FR07)
Q9	How much time per week do you or your team spend manually pulling together sales data rather than actually analysing it?	Quantifies business cost of the current gap; justifies system investment (NFR06)
Q10	If the system showed you that a skin category was underperforming in one region but performing well in another, would you act on that, and what would you need to see to feel confident in that data?	Surfaces report trustworthiness and granularity needs (FR09, NFR05)

2. Finance and Revenue Department

Questions directed at Finance Officers and Revenue Analysts responsible for forecasting MTX income and monitoring commercial performance across reporting periods.

Q#	Question (User Perspective)	Requirement Outcome Targeted
Q1	When you are preparing a quarterly revenue forecast, how do you currently get the MTX sales figures you need, and what are the biggest gaps in that data?	Establishes current-state pain; surfaces need for consolidated revenue data (FR10)
Q2	Have you ever had a league launch where the actual revenue came in significantly different from your forecast, and if so, what caused that gap?	Confirms need for trend-based forecasting support (FR10, NFR01)
Q3	Can you currently see what proportion of your MTX revenue comes from bundles versus individual item purchases on a given day, week, or month?	Confirms bundle/individual purchase distinction (FR05, FR10)
Q4	How do you currently measure whether a high-value spending cohort, such as your top spenders, is growing, stable, or shrinking over time?	Surfaces spending-tier trend analysis need (FR07, FR10)
Q5	When you need to produce a revenue report for leadership ahead of a new league, what manual steps are involved, and where does that process break down?	Identifies reporting friction and establishes case for automated reporting (FR09, FR12)
Q6	Do you have visibility into which geographic markets are contributing most to MTX revenue, or is that something you currently cannot see without IT support?	Confirms regional revenue visibility need (FR06, FR09)
Q7	Are there any compliance or audit requirements that mean you need an auditable record of every transaction that has been captured and reported?	Surfaces data integrity and audit trail needs (FR01, NFR03)
Q8	If you could see revenue broken down by both item category and character class in the same report, would that change how you approach budget allocation?	Confirms cross-dimensional reporting need (FR02, FR03, FR09)
Q9	What level of reporting access should a Finance Officer have compared to, say, a Data Analyst? Are there things you should not be able to see or change?	Confirms role-based access control requirement (FR13, FR14)
Q10	How quickly after a transaction is completed do you need to see it reflected in a report for it to be useful to your planning processes?	Confirms performance and latency requirement (NFR01, NFR05)

3. Game Development Department

Questions directed at Designers, Art Leads, and Producers responsible for commissioning, scheduling, and releasing new cosmetic content.

Q#	Question (User Perspective)	Requirement Outcome Targeted
Q1	When you are deciding which new cosmetic items to put into production next, what data do you currently look at, and do you feel that information is reliable?	Surfaces need for sales-driven content planning (FR04, FR11)
Q2	Have you ever invested design and art resources into a skin category that turned out not to perform well commercially? How did you find out it was underperforming?	Confirms underperformance flagging requirement (FR11); establishes cost of current gap
Q3	Do you currently have any way of knowing which character classes have the fewest cosmetic options relative to their active player population?	Confirms class-level coverage analysis need (FR03, FR04)
Q4	When a new league launches, are you able to see within the first few days which new cosmetic items are being purchased most frequently, or does that take weeks to surface?	Confirms near-real-time reporting need (NFR01, FR10)
Q5	If you could see a ranked list of the best-selling and worst-selling skins by character class, would that directly influence your content pipeline decisions?	Confirms class-specific best/worst seller ranking requirement (FR03, FR04, FR11)
Q6	Is there a formal process for deciding which character classes receive new cosmetics each league, or is it largely based on intuition and internal discussion?	Establishes that data-driven process is currently absent; supports system value proposition
Q7	When you are reviewing a skin that has been in the catalogue for several months, how do you judge whether it is worth promoting, discounting, or retiring?	Confirms need for item lifecycle performance tracking (FR04, FR08, FR11)
Q8	Do you have a way to compare how a new skin's sales trajectory in its first week compares to previous launches of similar item types?	Surfaces need for time-series comparative analysis (FR04, FR10)
Q9	Would it be useful for you to see, when reviewing a cosmetic item, whether it is primarily being purchased by high-spending players or by a broader casual segment?	Confirms cross-dimensional view of item performance by spending tier (FR07, FR09)
Q10	What report or dashboard view would you check every morning during a league launch if it existed and you trusted the data in it?	Elicits specific UI and reporting requirements; surfaces daily-use workflow needs (FR09, FR10, NFR06)

4. Data Analytics Department

Questions directed at Data Analysts and BI Specialists responsible for transforming raw transaction data into actionable organisational insight.

Q#	Question (User Perspective)	Requirement Outcome Targeted
Q1	When a stakeholder from Marketing or Finance asks you for a custom analysis, what is the first obstacle you usually run into in terms of getting the data you need?	Surfaces data access friction and pipeline gap (FR01, FR09)
Q2	Do you currently have access to a consistent, well-structured dataset that records every microtransaction with all relevant attributes, or are you piecing that together from multiple sources?	Confirms data fragmentation problem (FR01, FR02, FR03)
Q3	If you wanted to segment players by spending tier and then filter their purchases by item category and region simultaneously, could you do that today, and how long would it take?	Confirms multi-dimensional segmentation and filtering requirements (FR06, FR07, FR09)
Q4	Are there analytical questions that leadership has asked you that you simply cannot answer because the data does not exist or is not accessible in a usable form?	Identifies specific gaps that functional requirements must close
Q5	How do you currently handle the latency between a transaction occurring in the game and that transaction being available for you to analyse?	Confirms real-time or near-real-time data capture requirement (NFR01)
Q6	If you had a reliable, pre-aggregated dataset of all microtransaction events updated within two seconds of each purchase, what analyses would you run first?	Surfaces highest-priority analytical use cases driving FR04, FR10, FR11
Q7	Are there any data quality or consistency problems with the transaction data you currently work with, such as missing fields, inconsistent category labels, or duplicate records?	Surfaces data normalisation and classification requirements (FR02, FR03)
Q8	When you produce an analysis and share it with another department, do they ever question whether the numbers are accurate? What causes that distrust?	Confirms need for authoritative single source of truth; supports system rationale
Q9	Do you have concerns about how player data is currently being handled with respect to privacy obligations, and are there things you currently cannot do because of those constraints?	Surfaces privacy compliance and data governance requirements (NFR03)
Q10	If the system automatically flagged items below a sales threshold and sent an alert to the relevant team, would that reduce the amount of reactive work your team currently does?	Confirms automated alerting and threshold configuration requirement (FR11, NFR06)

2.2 Functional Requirements

Using the client questions from 2.1.1 the functional and non-functional requirements were derived. In the table below it shows the requirement and an identifier. FR represents functional requirement and the number besides it links the requirement to the ID. NFR represents non-functional requirements. The functional requirements have been placed into different sections which reflect what uses

2.2.1 Sales Capture and Transaction Tracking

ID	Requirement
FR01	The system shall record every microtransaction purchase event, capturing the item identifier, transaction price, timestamp, and associated player account.
FR02	The system shall categorise each purchase by item type, including weapon skins, armour skins, pets, portal effects, hideout decorations, and stash tab expansions.
FR03	The system shall record the character class active at the time of each purchase to enable class-correlated sales analysis.
FR04	The system shall identify the highest-selling microtransaction items over any user-selectable reporting period.
FR05	The system shall distinguish between bundle purchases and individual item purchases, recording each transaction type separately.

2.2.2 Player Demographics and Segmentation

ID	Requirement
FR06	The system shall record the player's geographic region or country at time of purchase to facilitate regional sales analysis.
FR07	The system shall classify player accounts by account tenure and spending tier: Casual (under \$50 lifetime spend), Regular (\$50–\$200), and High-Value (over \$200).
FR08	The system shall differentiate between first-time purchasers and repeat buyers on a per-item basis to support customer retention analysis.

2.2.3 Reporting, Analytics, and Export

ID	Requirement
FR09	The system shall generate sales reports filterable by date range, geographic region, item type, and character class.
FR10	The system shall display revenue trend visualisations across configurable time windows including daily, weekly, and monthly aggregations.
FR11	The system shall automatically flag microtransaction items whose sales performance falls below configurable underperformance thresholds.
FR12	The system shall permit authorised users to export report data in CSV, PDF, or JSON format for downstream use.

2.2.4 Role-Based Access and Security

ID	Requirement
FR13	The system shall support four distinct authenticated user roles: Data Analyst, Marketing Manager, Game Developer, and Finance Officer, each with access permissions appropriate to their departmental function.
FR14	The system shall restrict data export capabilities and system configuration functions to roles with explicitly authorised access, enforced through role-based access control.

2.3 Non-Functional Requirements

ID	Category	Requirement
NFR01	Performance	The system shall process and store each MTX purchase event within 2 seconds of transaction completion.
NFR02	Availability	The system shall maintain 99.9% uptime during peak league launch windows - the highest-demand periods for both gameplay and microtransactions.
NFR03	Legal / Privacy	All personally identifiable player data must be collected, stored, and processed in accordance with the New Zealand Privacy Act 2020.
NFR04	Scalability	The system shall support a minimum of 50 concurrent internal users without performance degradation.
NFR05	Performance	Dashboard reports shall render within 3 seconds when querying datasets of up to 1,000,000 records.
NFR06	Usability	The system interface shall be operable without formal training by staff already familiar with standard business dashboards.

2.4 Stakeholder Traceability Matrix

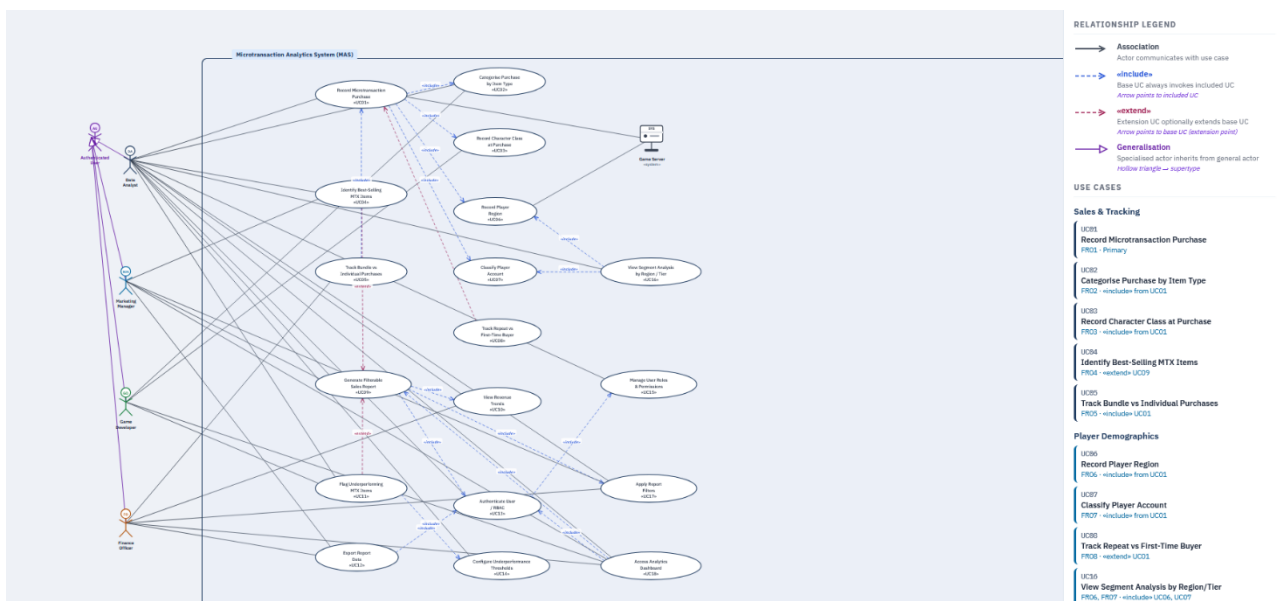
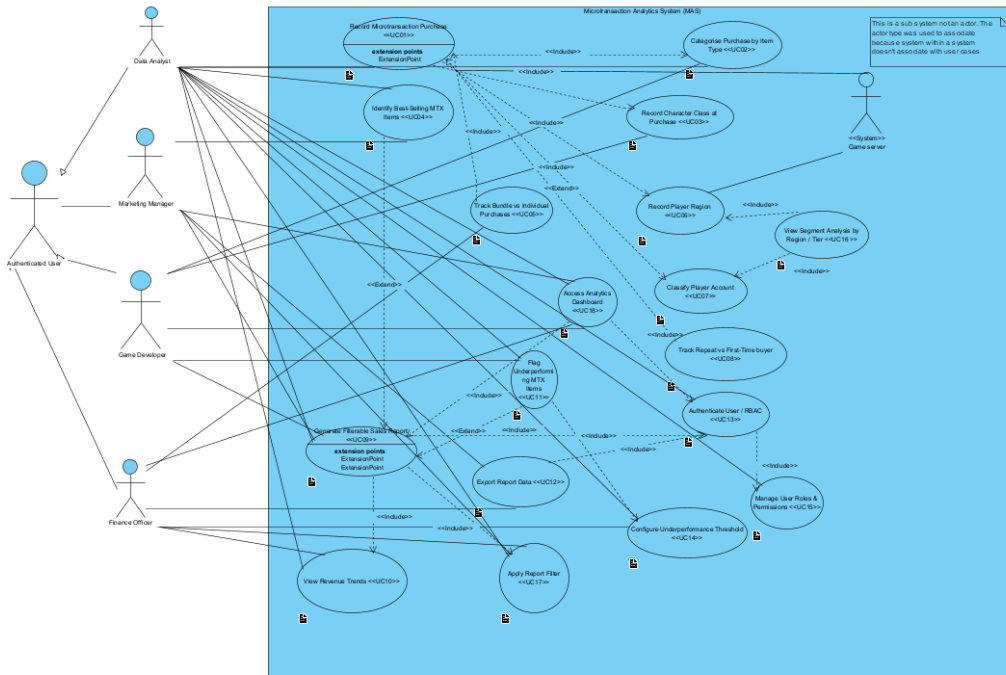
Below is a table linking the stakeholders to requirements

Stakeholder	Department	Related Requirements
Data Analyst	Data Analytics	FR01, FR02, FR03, FR04, FR05, FR06, FR07, FR08, FR09, FR10, FR11, FR12, FR13, FR14
Marketing Manager	Marketing	FR04, FR06, FR09, FR11, FR12, FR13, FR14
Finance Officer	Finance/Revenue	FR05, FR07, FR09, FR10, FR12, FR13, FR14
Game Developer	Game Development	FR02, FR03, FR04, FR11, FR13
Player (indirect)	External	FR06, FR07, FR08 (data subject - not a system user)

Task 3: UML Requirements Modelling

3.1 Use Case Diagram

The Use Case Diagram covers the full system scope of the MAS, with the system boundary encapsulating all fourteen use cases across the four functional requirement categories. Four internal actors: Data Analyst, Marketing Manager, Finance Officer, and Game Developer - interact with the system. The diagram incorporates include, extend, and generalisation relationships as described below. There is both visual paradigm and html versions.



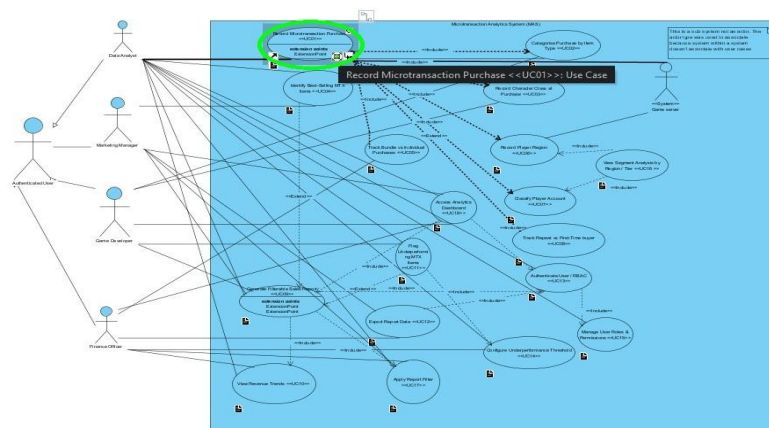
3.2 Fully Dressed Use Case Descriptions: Use cases explained

Technical table of use cases explained for all 18 cases as well as a quick summary of each use case

UC01: Record Microtransaction Purchase

Field	Description
Use Case ID	UC01
Use Case Name	Record Microtransaction Purchase
Actor(s)	Data Analyst (primary); TransactionService (system actor)
Description	The system captures a new microtransaction purchase event when a player completes a purchase in-game, recording all required attributes for analytics processing.
Preconditions	Player account exists in the system. The purchased item exists in the MTX catalogue. Game server transaction is confirmed.
Main Success Flow	1. Game server notifies MAS of a completed purchase event. 2. System validates required fields: item ID, price, timestamp, player account ID, character class. 3. System creates a Transaction record with a unique GUID. 4. System increments the item's sales counter. 5. System records purchase in the player's history and updates spending tier. 6. System persists the transaction to the repository. 7. System confirms successful capture.
Alternate Flow	A2 (Step 3): Item ID not found in catalogue - transaction rejected; error logged; player refunded.
Exception Flow	E1: Repository unavailable - transaction queued for retry; NFR02 breach alert raised.
Postconditions	A new Transaction record exists in the repository. The item's SalesCount is incremented. The player's purchase history is updated.
«includes»	«include» Validate Transaction Data
«extends»	None

UC 1 Position

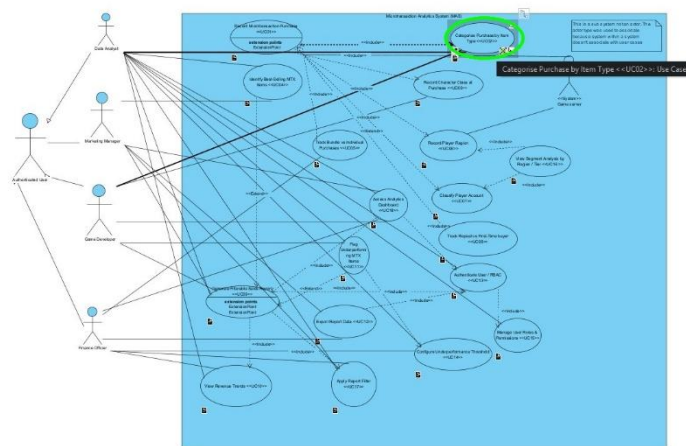


Use Case	Name	Summary
UC01	Record Microtransaction Purchase	Captures and persists a completed in-game purchase event, recording all required attributes (item, price, timestamp, player, character class) to the transaction repository.

UC02: Generate Sales Report

Field	Description
Use Case ID	UC02
Use Case Name	Generate Sales Report
Actor(s)	Data Analyst, Marketing Manager, Finance Officer, Game Developer
Description	An authenticated user configures filter criteria and requests the system to generate an analytics report across the filtered dataset.
Preconditions	User is authenticated with a valid role. Transaction data exists in the repository. Filter criteria are valid (start date before end date).
Main Success Flow	1. User selects desired filter criteria: date range, region, item type, character class. 2. System validates FilterSet object. 3. System applies filters to the transaction repository. 4. ReportEngine calculates: total revenue, top sellers by category and class, revenue trend, bundle split, tier distribution, underperforming items. 5. System populates a Report DTO with all computed values. 6. System renders the report in the requested dashboard view. 7. User reviews the report.
Alternate Flow	A2 (Step 3): No transactions match the filter - system displays an empty report with a notification.
Exception Flow	E1: DateRange invalid (end before start): system rejects request and prompts user to correct dates.
Postconditions	A Report object is generated and displayed. No data is modified.
«includes»	«include» Authenticate User
«extends»	«extend» Export Report (if user requests export)

UC 2 Position

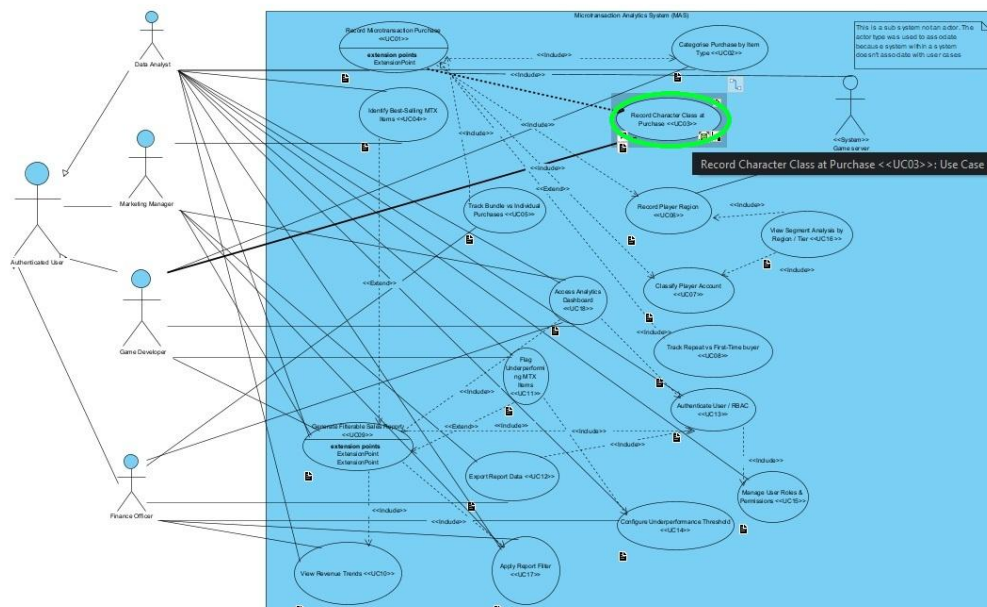


Use Case	Name	Summary
UC02	Generate Sales Report	Allows an authenticated user to apply filter criteria and generate a comprehensive analytics report showing revenue, top sellers, trends, and item performance across the filtered dataset.

UC03: Export Report

Field	Description
Use Case ID	UC03
Use Case Name	Export Report
Actor(s)	Marketing Manager, Finance Officer, Data Analyst
Description	An authorised user exports a generated report to a file in CSV, PDF, or JSON format for use in downstream activities such as campaign planning or financial forecasting.
Preconditions	User is authenticated. User role has CanExport() permission (Marketing Manager, Finance Officer, or Analyst). A report has been generated.
Main Success Flow	1. User selects desired export format (CSV, PDF, or JSON). 2. System verifies user role has export permission. 3. ExportService instantiated with selected format. 4. System serialises the Report DTO to the chosen format. 5. System writes the file to the specified output path. 6. System confirms export completion to user.
Alternate Flow	A2 (Step 2): User role does not have export permission - system displays an access denied message; no file is written.
Exception Flow	E1: File system write error : system logs error and notifies user; no partial file persisted.
Postconditions	A file in the selected format exists at the specified path. The original report data is unchanged.
«includes»	«include» Authenticate User; «include» Verify Export Permission
«extends»	None : this use case is itself extended by Generate Sales Report

UC 3 Position

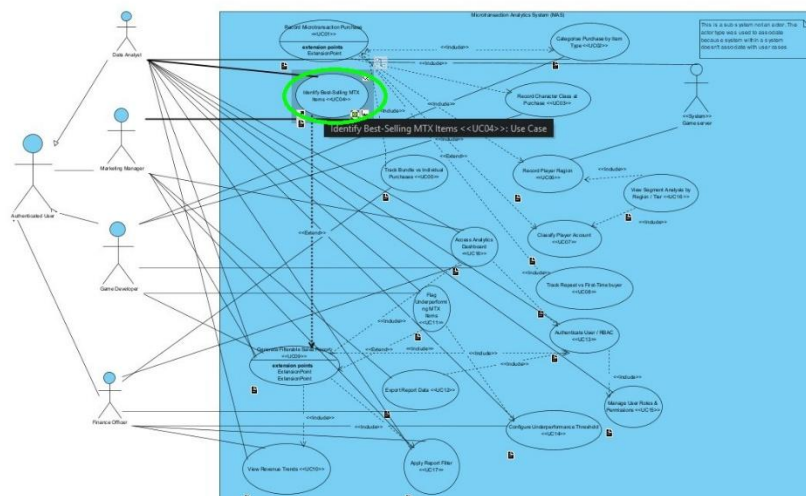


Use Case	Name	Summary
UC03	Export Report	Enables an authorised user to serialise a generated report into a chosen file format (CSV, PDF, or JSON) and save it for use in downstream activities such as campaign planning or financial forecasting.

UC04: Flag Underperforming Items

Field	Description
Use Case ID	UC04
Use Case Name	Flag Underperforming Items
Actor(s)	Data Analyst (configures threshold); Game Developer, Marketing Manager (view flags)
Description	The system automatically compares each item's cumulative sales count against a configurable threshold and flags those that fall below it for review by relevant stakeholders.
Preconditions	An underperformance threshold has been configured by an Analyst or Developer. Transaction data covering the relevant period exists.
Main Success Flow	1. ReportEngine retrieves all items from the catalogue. 2. System calculates total sales count per item from the transaction repository. 3. System compares each item's count against the configured threshold. 4. Items below threshold are added to the UnderperformingItems list in the Report DTO. 5. Flagged items are surfaced in the Underperforming dashboard view. 6. Relevant stakeholders (Developer, Marketing) receive the alert list.
Alternate Flow	A1: No items fall below threshold: system confirms all items are performing above threshold.
Exception Flow	E1: Threshold not configured - system uses default value of 15 units; logs warning to Analyst.
Postconditions	Underperforming items are identified and available in report output. No item data is modified.
«includes»	«include» Generate Sales Report
«extends»	None

UC 4 Position

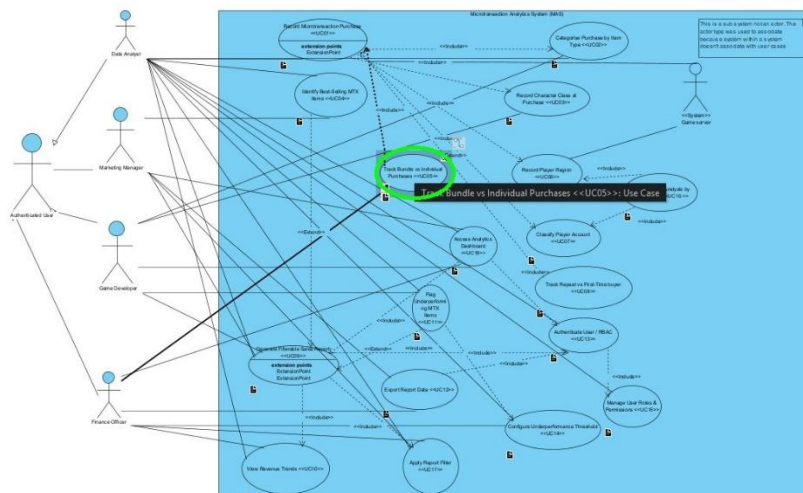


Use Case	Name	Summary
UC04	Flag Underperforming Items	Automatically compares each catalogue item's sales count against a configurable threshold and surfaces any items falling below it to relevant stakeholders for review.

UC05: Authenticate User / Login

Field	Description
Use Case ID	UC05
Use Case Name	Authenticate User
Actor(s)	Any internal staff user (Data Analyst, Marketing Manager, Finance Officer, Game Developer)
Description	A staff user provides credentials to authenticate with the MAS. On success the system grants role-appropriate access; on failure the attempt is rejected without revealing whether the username or password was incorrect.
Preconditions	The user account exists in the system. The user is not currently authenticated.
Main Success Flow	1. User enters username and password on the Login form. 2. AuthService searches the user list for a matching username (case-insensitive). 3. System computes SHA-256 hash of the provided password. 4. System compares hash against the stored hash for the matching account. 5. On match: CurrentUser is set; session is established; user is directed to the main dashboard. 6. Dashboard is rendered with views and actions appropriate to the user's role.
Alternate Flow	A1 (Step 2–4): Credentials invalid - system returns a generic rejection message; CurrentUser remains null; login form remains open.
Exception Flow	E1: User closes login window without authenticating - application exits.
Postconditions	Success: User session active; role-based capabilities available. Failure: No session created; credentials not disclosed in error messages.
«includes»	None - this use case is included by all other use cases
«extends»	None
Generalisation	All four internal actor types (Analyst, Marketing Manager, Finance Officer, Developer) generalise from a base Staff User actor for this use case.

UC 5 Position

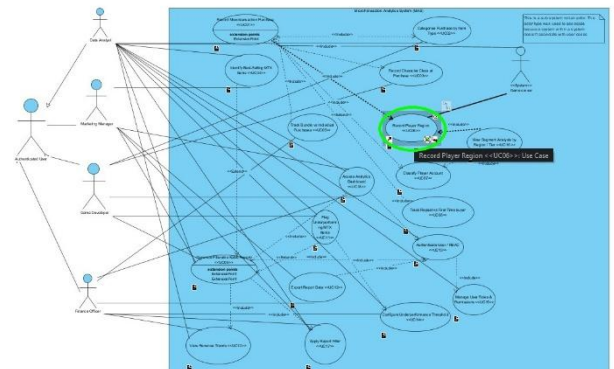


Use Case	Name	Summary
UC05	Authenticate User / Login	Validates staff credentials against stored password hashes and, on success, establishes a session and grants role-appropriate access to the system.

UC06: Record Player Region

Field	Description
Use Case ID	UC06
Use Case Name	Record Player Region
Actor(s)	Game Server (primary system actor); TransactionService
Description	When a microtransaction purchase event is received, the system automatically records the player's geographic region or country at the exact time of purchase. This data enriches the transaction record to enable regional segmentation analysis.
Preconditions	A transaction event has been received from the Game Server (UC01 is executing). The player account exists in the system. Region metadata is available from the player's account profile or IP geolocation service.
Main Success Flow	1. Game Server transmits a purchase event containing the player account ID. 2. System queries the player's account profile for the registered region attribute. 3. If region is not in the account profile, system falls back to IP geolocation lookup. 4. System validates the resolved region against the supported RegionEnum values (e.g., NZ, AU, US, EU, ASIA). 5. System assigns the region value to the RegionCode field in the Transaction record. 6. Transaction record is persisted with the region field populated. 7. Region data becomes available for segmentation queries.
Alternate Flow	A1 (Step 2–3): Region cannot be resolved from profile or geolocation - system assigns RegionCode.UNKNOWN and logs a warning; transaction is not rejected.
Exception Flow	E1: Geolocation service is unavailable - system uses the account's last known region; NFR02 latency threshold is still monitored.
Postconditions	The Transaction record contains a valid RegionCode value. The data is available for use in UC16 (View Segment Analysis by Region / Tier).
«includes»	«include» Record Microtransaction Purchase (UC01) : this UC executes as part of UC01's flow
«extends»	None: UC06 is included by UC01 and UC16

UC 6 Position

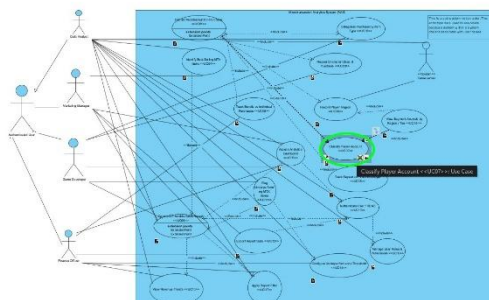


Use Case	Name	Summary
UC06	Record Player Region	Automatically resolves and attaches a player's geographic region to each transaction record at the time of purchase, enabling regional segmentation analysis.

UC07: Classify Player Account

Field	Description
Use Case ID	UC07
Use Case Name	Classify Player Account
Actor(s)	Game Server (system actor); TransactionService
Description	The system classifies each player account into a spending tier (Casual, Regular, or Whale) based on cumulative lifetime spending and account age. Classification is updated each time a new transaction is recorded and is stored on the PlayerAccount object.
Preconditions	A transaction event is being processed (UC01 is executing). The player account record exists and contains a LifetimeSpend value. Spending tier thresholds are configured in the system (default: Casual < \$50, Regular \$50–\$200, Whale > \$200).
Main Success Flow	1. On receiving a new transaction, system retrieves the player's current LifetimeSpend from the PlayerAccount record. 2. System adds the current transaction amount to LifetimeSpend. 3. System evaluates the updated LifetimeSpend against the configured tier thresholds. 4. System sets the player's SpendingTier attribute to the appropriate SpendingTierEnum value (CASUAL, REGULAR, or WHALE). 5. System records the AccountAge (in days since account creation). 6. Updated PlayerAccount object is persisted to the repository. 7. SpendingTier is stamped on the Transaction record for reporting.
Alternate Flow	A1 (Step 3): LifetimeSpend exactly equals a threshold boundary - system assigns the higher tier.
Exception Flow	E1: PlayerAccount record not found : transaction is queued; system raises an alert; account data inconsistency is logged for Analyst review.
Postconditions	The player's SpendingTier is updated in the PlayerAccount record. The Transaction record carries the current SpendingTier at time of purchase. Classification data is available for UC16 (segment analysis).
«includes»	«include» Record Microtransaction Purchase (UC01) - executes as part of UC01's flow
«extends»	None: UC07 is included by UC01 and UC16

UC 7 Position

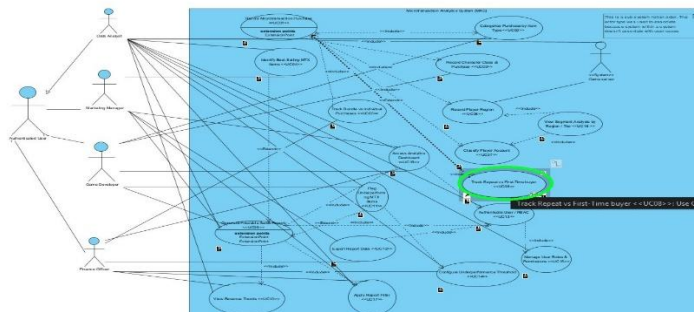


Use Case	Name	Summary
UC07	Classify Player Account	Updates a player's spending tier (Casual, Regular, or Whale) after each purchase by evaluating cumulative lifetime spend against configurable thresholds, and stamps the tier on the transaction record.

UC08: Track Repeat vs First-Time Buyer

Field	Description
Use Case ID	UC08
Use Case Name	Track Repeat vs First-Time Buyer
Actor(s)	TransactionService (system actor)
Description	When a transaction is recorded, the system determines whether the purchasing player has previously purchased the same item. This distinguishes first-time buyers from repeat purchasers per item, enriching the transaction record for retention and loyalty analysis.
Preconditions	A new transaction is being processed (UC01 is executing). The player's purchase history is accessible in the repository. The item ID is present in the current transaction.
Main Success Flow	1. System retrieves the player's purchase history from the repository, filtered by the current item ID. 2. System checks whether a prior transaction record exists for the same player-item combination. 3. If no prior record exists, system sets the PurchaseType flag to FIRST_TIME on the Transaction record. 4. If a prior record exists, system sets the PurchaseType flag to REPEAT. 5. System increments the item's RepeatBuyerCount or FirstTimeBuyerCount metric accordingly. 6. The Transaction record with the PurchaseType flag is persisted.
Alternate Flow	A1 (Step 1): Purchase history query returns no results (new player account) - system treats the transaction as FIRST_TIME automatically.
Exception Flow	E1: Purchase history repository is temporarily unavailable - system defaults PurchaseType to FIRST_TIME with a warning flag; Analyst is notified for manual review.
Postconditions	The Transaction record contains a PurchaseType value (FIRST_TIME or REPEAT). Item-level buyer-type counters are updated. Data is available for loyalty and retention reports.
«includes»	None - UC08 is an extension of UC01
«extends»	«extend» Record Microtransaction Purchase (UC01) - triggered when the purchase event is processed and player history is available

UC 8 Position



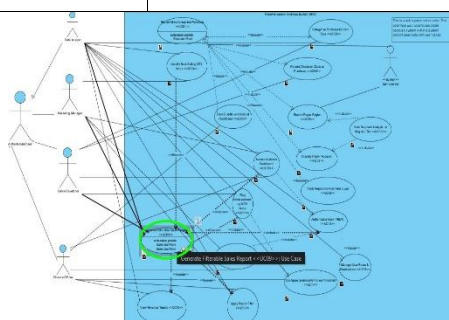
Use Case	Name	Summary
UC08	Track Repeat vs First-Time Buyer	Determines whether a player has previously purchased the same item and tags each transaction as either a first-time or repeat purchase to support retention and loyalty analysis.

UC09: Generate Filterable Sales Report

Field	Description
Use Case ID	UC09
Use Case Name	Generate Filterable Sales Report
Actor(s)	Data Analyst (primary); Marketing Manager; Finance Officer
Description	An authenticated user specifies a FilterSet comprising one or more of: date range, player region, item type, and character class. The system applies the filters to the transaction repository and presents a comprehensive sales report in the analytics dashboard.
Preconditions	User is authenticated with a valid role (UC13 has been completed). Transaction data exists in the repository for the requested period. FilterSet parameters are syntactically valid.
Main Success Flow	1. User selects desired filter criteria: DateRange, RegionCode, ItemType, CharacterClass. 2. System validates the FilterSet object (start date $\leq$ end date; valid enum values). 3. UC17 (Apply Report Filters) is invoked to apply the FilterSet to the transaction repository. 4. UC10 (View Revenue Trends) is invoked to compute trend data for the filtered dataset. 5. ReportEngine calculates: total revenue, top sellers, bundle vs individual split, tier distribution, and underperforming item flags. 6. System populates a Report DTO with all computed values. 7. System renders the report in the AnalyticsDashboard (UC18). 8. User reviews the rendered report.
Alternate Flow	A1 (Step 3): No transactions match the applied filters -system renders an empty report view with a notification message; no error is raised.
Exception Flow	E1: DateRange is invalid (end before start) - system rejects the request, highlights the offending fields, and prompts the user to correct the input. E2: ReportEngine throws an exception -system displays an error message; partial results are not shown; error is logged.
Postconditions	A Report DTO is generated and rendered in the dashboard. No transaction data is modified. The report is available for export via UC12.
«includes»	«include» Authenticate User / RBAC (UC13) «include» Apply Report Filters (UC17) «include» View Revenue Trends (UC10)
«extends»	«extend» Identify Best-Selling MTX Items (UC04) - extends UC09 when best-seller analysis is triggered «extend» Flag Underperforming MTX Items (UC11) - extends UC09 when underperformance flags are rendered

Use Case	Name	Summary
UC09	Generate Filterable Sales Report	Lets an authenticated user build a filtered sales report by specifying date range, region, item type, and character class, then renders the results - including trends, best sellers, and underperformance flags - in the analytics dashboard.

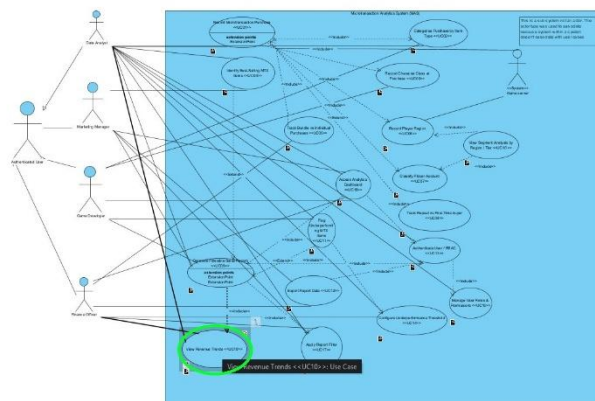
UC 9 Position



UC10: View Revenue Trends

Field	Description
Use Case ID	UC10
Use Case Name	View Revenue Trends
Actor(s)	Data Analyst (primary); Finance Officer
Description	The system calculates and displays revenue trend data over configurable time windows (daily, weekly, monthly) for the currently filtered dataset. Trend data is rendered as a time-series view within the analytics dashboard to support forecasting and performance review.
Preconditions	A sales report has been generated by UC09 (Generate Filterable Sales Report). User is authenticated with a valid role. Transaction data exists for the selected time window.
Main Success Flow	1. System receives the filtered transaction dataset from UC09's ReportEngine. 2. User selects the desired granularity: DAILY, WEEKLY, or MONTHLY. 3. System groups transactions into time buckets according to the selected granularity. 4. System sums revenue per bucket and constructs a TrendDataPoint list. 5. System computes period-over-period percentage change for each consecutive bucket pair. 6. System populates the RevenueTrends field in the Report DTO. 7. The trend data is rendered as a time-series chart in the dashboard.
Alternate Flow	A1 (Step 3): Fewer than two time buckets contain data - system renders available data with a note that trend comparison is not possible.
Exception Flow	E1: Insufficient data for the selected granularity (e.g., DAILY selected but only one day of data) - system defaults to WEEKLY and notifies the user.
Postconditions	The Report DTO contains a populated RevenueTrends list. Trend chart is displayed in the dashboard view. No transaction data is modified.
«includes»	None - UC10 is included by UC09
«extends»	None

UC 10 Position

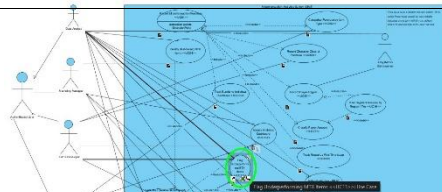


Use Case	Name	Summary
UC10	View Revenue Trends	Calculates and displays period-over-period revenue changes at daily, weekly, or monthly granularity for the currently filtered dataset as a time-series chart in the dashboard.

UC11: Flag Underperforming MTX Items

Field	Description
Use Case ID	UC11
Use Case Name	Flag Underperforming MTX Items
Actor(s)	Data Analyst (configures threshold); Game Developer; Marketing Manager (view flags)
Description	The system automatically evaluates each item's cumulative sales count against a configurable underperformance threshold and flags items that fall below it. Flagged items are surfaced in a dedicated dashboard view and can trigger notifications to relevant stakeholders for review and remedial action.
Preconditions	A sales report has been generated by UC09. An underperformance threshold has been configured via UC14; if not, the system default (15 units per reporting period) is used. The transaction repository contains data for the relevant reporting period.
Main Success Flow	1. ReportEngine retrieves all active items from the MTX catalogue. 2. System calculates total SalesCount per item from the filtered transaction dataset. 3. System retrieves the configured underperformance threshold from ThresholdConfig. 4. System compares each item's SalesCount against the threshold. 5. Items below threshold are added to the UnderperformingItems list in the Report DTO. 6. Flagged items are rendered in the Underperforming dashboard view with their sales count and threshold delta. 7. UC14 (Configure Underperformance Thresholds) may be invoked if the Analyst wishes to revise the threshold inline. 8. Relevant stakeholders (Developer, Marketing Manager) receive the flag list.
Alternate Flow	A1 (Step 4): No items fall below the threshold - system confirms all items are performing at or above threshold; the Underperforming view displays an empty list with a confirmation message.
Exception Flow	E1: ThresholdConfig not found - system applies the default threshold of 15 units and logs a warning to the Data Analyst. E2: MTX catalogue is empty - system skips the evaluation and notifies the Analyst.
Postconditions	Underperforming items are identified and listed in the Report DTO. Flagged items are visible in the dashboard. No item catalogue data is modified.
«includes»	«include» Configure Underperformance Thresholds (UC14)
«extends»	«extend» Generate Filterable Sales Report (UC09) - extends UC09 when underperformance analysis is rendered

UC 11 Position

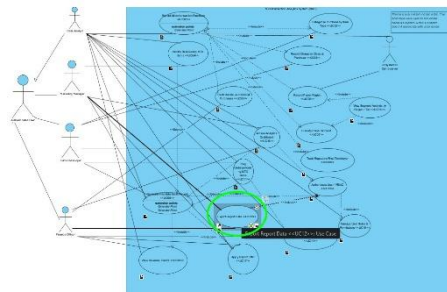


Use Case	Name	Summary
UC11	Flag Underperforming MTX Items	Evaluates every active catalogue item's sales count against a configurable threshold and lists underperforming items in a dedicated dashboard view for review by analysts, developers, and marketing.

UC12: Export Report Data

Field	Description
Use Case ID	UC12
Use Case Name	Export Report Data
Actor(s)	Data Analyst; Marketing Manager; Finance Officer
Description	An authenticated user with export permission requests the system to serialise a generated Report DTO to a chosen file format (CSV, PDF, or JSON). The ExportService produces the file and makes it available for download or saving to a specified output path.
Preconditions	User is authenticated and has been verified via UC13. The user role has CanExport() permission (Data Analyst, Marketing Manager, or Finance Officer). A Report DTO has been generated by UC09.
Main Success Flow	1. User selects the desired export format: CSV, PDF, or JSON. 2. System verifies that the user's role has the CanExport() permission. 3. ExportService is instantiated with the selected ExportFormat enum value. 4. System serialises the Report DTO to the chosen format. 5. System writes the output file to the designated export path. 6. System confirms export completion to the user and provides a download link or file path.
Alternate Flow	A1 (Step 2): User role does not have CanExport() permission - system displays an access-denied message; no file is written; the event is logged for audit purposes.
Exception Flow	E1: File system write error - system logs the error, notifies the user, and ensures no partial file is persisted. E2: Report DTO is null or empty - system rejects the export request and prompts the user to generate a report first.
Postconditions	An export file in the selected format exists at the designated output path. The original Report DTO is unchanged. Export action is logged for audit purposes.
«includes»	«include» Authenticate User / RBAC (UC13) «include» Verify Export Permission
«extends»	None - UC12 is itself extended by UC09 (Export Report is an extension point of Generate Sales Report)

UC 12 Position

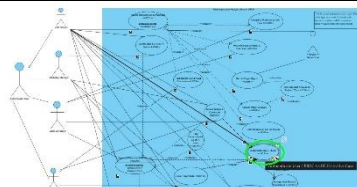


Use Case	Name	Summary
UC12	Export Report Data	Serialises a generated Report DTO to CSV, PDF, or JSON and delivers the file to an authorised user, while logging the export action for audit purposes.

UC13: Authenticate User / RBAC

Field	Description
Use Case ID	UC13
Use Case Name	Authenticate User / RBAC
Actor(s)	Any internal staff user: Data Analyst, Marketing Manager, Finance Officer, Game Developer
Description	A staff user provides credentials (username and password) to authenticate with the MAS. On successful authentication, the system establishes a session and enforces role-based access control (RBAC), granting access only to views and actions permitted for the user's assigned role.
Preconditions	The user account exists in the system user list. The user is not currently authenticated. The MAS login interface is available.
Main Success Flow	1. User enters username and password on the login form. 2. AuthService retrieves the matching UserAccount from the user list (case-insensitive username match). 3. System computes the SHA-256 hash of the provided password. 4. System compares the computed hash against the stored PasswordHash on the UserAccount. 5. On match: CurrentUser is set; a session is established; the user's Role is loaded from the UserAccount. 6. UC15 (Manage User Roles & Permissions) is invoked to load the access-control matrix for the user's role. 7. The AnalyticsDashboard is rendered with views and actions filtered to those permitted by the user's role.
Alternate Flow	A1 (Steps 2–4): Credentials are invalid : system returns a generic rejection message without disclosing whether the username or password was incorrect; CurrentUser remains null; the login form remains open.
Exception Flow	E1: User closes the login window without authenticating - application exits. E2: AuthService is unavailable - system displays a service-unavailable message; no session is created.
Postconditions	Success: Active session exists; CurrentUser is set; role-based capabilities are enforced. Failure: No session is created; credentials are not disclosed in error messages.
«includes»	«include» Manage User Roles & Permissions (UC15)
«extends»	None - UC13 is included by UC09, UC12, and UC18
Generalisation	Data Analyst, Marketing Manager, Finance Officer, and Game Developer all generalise from the base Authenticated User actor for this use case.

UC 13 Position

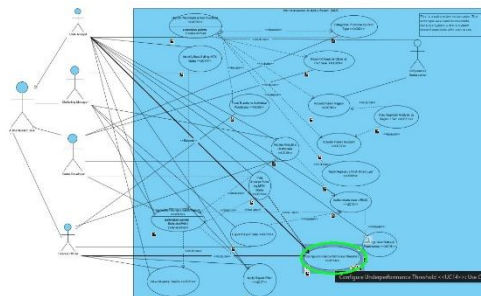


Use Case	Name	Summary
UC13	Authenticate User / RBAC	Authenticates a staff user via SHA-256 password verification, establishes a session, and enforces role-based access control so each user sees only the views and actions permitted for their role.

UC14: Configure Underperformance Thresholds

Field	Description
Use Case ID	UC14
Use Case Name	Configure Underperformance Thresholds
Actor(s)	Data Analyst (primary - authorised to configure); Game Developer (view only)
Description	An authorised Data Analyst configures the numeric threshold used by UC11 (Flag Underperforming MTX Items) to identify underperforming items. The threshold specifies the minimum acceptable sales count per item within a reporting period. Changes are persisted to the ThresholdConfig object and take effect on the next report generation.
Preconditions	User is authenticated via UC13 and has the ANALYST role. The ThresholdConfig object exists in the system (populated with a default value of 15 units if not previously set).
Main Success Flow	- Analyst navigates to the Threshold Configuration panel. - System displays the current threshold value from ThresholdConfig. - Analyst enters a new integer threshold value (must be > 0). - System validates the input: must be a positive integer within the permitted range (1–10,000). - System updates the ThresholdConfig object with the new value and timestamps the change. - System persists the updated ThresholdConfig to the repository. - System confirms the update to the Analyst and logs the change with the Analyst's user ID.
Alternate Flow	A1 (Step 4): Input value is not a positive integer or exceeds the maximum - system rejects the input with a validation message and retains the existing threshold.
Exception Flow	E1: Repository write fails - system retains the in-memory ThresholdConfig change for the current session and notifies the Analyst that the change was not persisted; a retry option is presented.
Postconditions	The ThresholdConfig record is updated with the new threshold value. The change is logged with the Analyst's identity and a timestamp. The next execution of UC11 uses the updated threshold.
«includes»	None - UC14 is included by UC11
«extends»	None

UC 14 Position

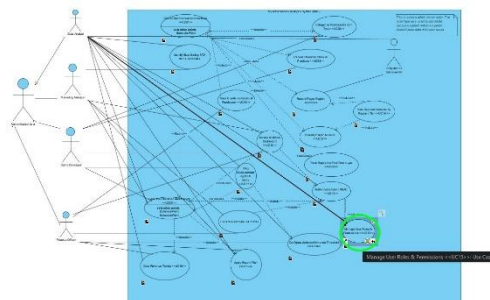


Use Case	Name	Summary
UC14	Configure Underperformance Thresholds	Allows an authorised Data Analyst to set the minimum sales count threshold used to identify underperforming items, with all changes persisted and timestamped for audit.

UC15: Manage User Roles & Permissions

Field	Description
Use Case ID	UC15
Use Case Name	Manage User Roles & Permissions
Actor(s)	Data Analyst (role administration); AuthService (system actor)
Description	The system loads and enforces the role-based access control matrix for an authenticated user. Separately, an authorised Data Analyst can assign or modify user role assignments to control which views and actions each staff member can access within the MAS.
Preconditions	UC13 (Authenticate User) has been completed and a valid session exists. The RBAC permission matrix is defined in the system for the four roles: DATA_ANALYST, MARKETING_MANAGER, FINANCE_OFFICER, GAME_DEVELOPER.
Main Success Flow	1. AuthService retrieves the authenticated user's assigned Role from the UserAccount. 2. System loads the RolePermissionMatrix for the user's Role (defines permitted views and actions). 3. System applies the permission matrix to the AnalyticsDashboard, enabling or disabling views and controls accordingly. 4. (Administration sub-flow) Analyst navigates to the User Administration panel. 5. Analyst selects a target UserAccount and assigns or updates the Role enum value. 6. System validates that the new role is a permitted RoleEnum value. 7. System persists the updated UserAccount role to the repository. 8. System logs the role change with the Analyst's identity and a timestamp.
Alternate Flow	A1 (Step 5): Analyst attempts to assign a role to their own account - system rejects the action to prevent privilege escalation.
Exception Flow	E1: RolePermissionMatrix not found for a given role - system denies all access for that role and raises an alert for the Analyst.
Postconditions	The active user's session reflects their permitted role capabilities. Any role assignment changes are persisted and audited. The dashboard displays only role-appropriate views and actions.
«includes»	None - UC15 is included by UC13
«extends»	None

UC 15 Position

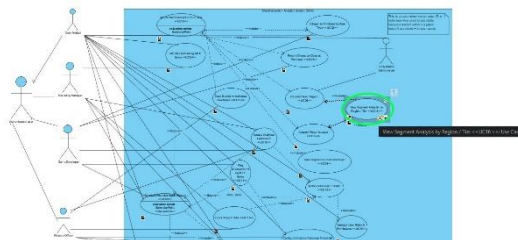


Use Case	Name	Summary
UC15	Manage User Roles & Permissions	Loads and enforces the role-based permission matrix for each authenticated user, and allows a Data Analyst to assign or update role assignments for other staff accounts.

UC16: View Segment Analysis by Region / Tier

Field	Description
Use Case ID	UC16
Use Case Name	View Segment Analysis by Region / Tier
Actor(s)	Data Analyst (primary); Marketing Manager
Description	An authenticated user requests a segmented view of purchase data broken down by player geographic region and/or spending tier. The system retrieves pre-recorded RegionCode and SpendingTier attributes from the transaction repository and aggregates them into a segmentation report for demographic and geographic analysis.
Preconditions	User is authenticated via UC13. Transaction records exist with populated RegionCode (from UC06) and SpendingTier (from UC07) fields. A date range filter is applied or defaults to the last 30 days.
Main Success Flow	1. User navigates to the Segment Analysis view in the AnalyticsDashboard. 2. User selects the segmentation dimension(s): by Region, by Tier, or both. 3. System invokes UC06 data (RegionCode per transaction) to group transactions by region. 4. System invokes UC07 data (SpendingTier per transaction) to group transactions by tier. 5. System calculates aggregate metrics per segment: total revenue, transaction count, average order value. 6. System populates the SegmentAnalysis section of the Report DTO. 7. Results are rendered as grouped bar charts and a summary table in the dashboard.
Alternate Flow	A1 (Step 3–4): No transactions contain RegionCode or SpendingTier data for the selected period - system notifies the user and renders an empty segmentation view.
Exception Flow	E1: Aggregation query exceeds the NFR02 performance threshold (3 seconds) - system displays a loading indicator; if the query exceeds 10 seconds, it is aborted and an error is reported.
Postconditions	Segment analysis data is displayed in the dashboard. Report DTO is updated with segmentation metrics. No transaction data is modified.
«includes»	«include» Record Player Region (UC06) - region data is sourced from UC06 «include» Classify Player Account (UC07) - tier data is sourced from UC07
«extends»	None

UC 16 Position

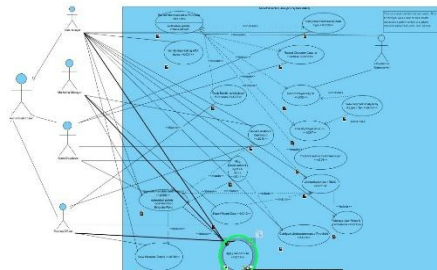


Use Case	Name	Summary
UC16	View Segment Analysis by Region / Tier	Aggregates transaction data by player geographic region and spending tier to produce a segmented revenue and volume breakdown, rendered as grouped charts and a summary table in the dashboard.

UC17: Apply Report Filters

Field	Description
Use Case ID	UC17
Use Case Name	Apply Report Filters
Actor(s)	Data Analyst; Marketing Manager; Finance Officer
Description	The system applies a user-defined FilterSet to the transaction repository to narrow the dataset used for report generation. The FilterSet encapsulates date range, player region, item type, and character class parameters. This use case maps to the ReportEngine.applyFilter() method and the FilterSet / DateRange classes in the domain model.
Preconditions	User is authenticated via UC13. A FilterSet object has been constructed from the user's filter selections in UC09. The transaction repository is accessible.
Main Success Flow	1. System receives a FilterSet object from the calling use case (UC09). 2. System validates the FilterSet: DateRange start $\leq$ end; all enum values are valid. 3. System calls ReportEngine.applyFilter(filterSet) to apply each filter dimension in sequence: a. Apply DateRange filter - select transactions within the specified date window. b. Apply RegionCode filter - if specified, restrict to the selected region(s). c. Apply ItemType filter - if specified, restrict to the selected item category. d. Apply CharacterClass filter - if specified, restrict to transactions by the selected class. 4. System returns the filtered transaction dataset to the calling use case (UC09). 5. The filtered dataset is used by the ReportEngine to calculate report metrics.
Alternate Flow	A1 (Step 3): One or more filter dimensions produce an empty intersection - system proceeds with the empty result set; the calling use case handles the empty report notification.
Exception Flow	E1: FilterSet is null or malformed - system raises a validation exception; UC09 receives the exception and prompts the user to correct the filter criteria.
Postconditions	A filtered transaction dataset is returned to UC09. No transaction data is modified. Filter parameters applied are recorded in the Report DTO metadata.
«includes»	None - UC17 is included by UC09
«extends»	None

UC 17 Position



Use Case	Name	Summary
UC17	Apply Report Filters	Applies a user-defined FilterSet (date range, region, item type, character class) to the transaction repository and returns the narrowed dataset to the report engine for metric calculation.

UC18: Access Analytics Dashboard

Field	Description
Use Case ID	UC18
Use Case Name	Access Analytics Dashboard
Actor(s)	Data Analyst; Marketing Manager; Finance Officer; Game Developer
Description	An authenticated user accesses the AnalyticsDashboard, the central interface of the MAS. The dashboard renders role-appropriate views including sales summaries, revenue trends, demographic segments, and underperforming item flags. Dashboard access requires valid authentication and loads all relevant sub-views through the included use cases.
Preconditions	User has successfully authenticated via UC13. The user's role has been loaded and the RBAC permission matrix has been applied via UC15. Transaction data exists in the repository.
Main Success Flow	1. User session is validated on dashboard load; UC13 is confirmed as active. 2. System determines permitted dashboard views based on the user's role (ViewTypeEnum). 3. System invokes UC09 (Generate Filterable Sales Report) to populate the default sales view with a default 30-day filter. 4. Dashboard renders the following role-appropriate view panels: a. Sales Summary - total revenue, top sellers, bundle split (all roles). b. Revenue Trends - time-series chart (Data Analyst, Finance Officer). c. Segment Analysis - region and tier breakdown (Data Analyst, Marketing Manager). d. Underperforming Items - flagged item list (Data Analyst, Game Developer, Marketing Manager). 5. User may adjust filter criteria, triggering UC17 and refreshing the dashboard. 6. User may initiate an export, triggering UC12.
Alternate Flow	A1 (Step 3): No transaction data exists for the default 30-day window - system renders the dashboard with empty view panels and a prompt to adjust the filter range.
Exception Flow	E1: Session has expired - system redirects the user to the login form; all unsaved filter selections are discarded. E2: Dashboard rendering fails for a specific view panel - system displays the available panels and shows an error indicator for the failed panel without blocking the rest of the dashboard.
Postconditions	The dashboard is rendered with role-appropriate views populated. The user can interact with filters, drill into sub-views, and initiate exports. No data is modified by dashboard access alone.
«includes»	«include» Authenticate User / RBAC (UC13) «include» Generate Filterable Sales Report (UC09)
«extends»	None



UC 18 Position

Use Case	Name	Summary
UC18	Access Analytics Dashboard	Provides authenticated users with a central, role-tailored interface that displays sales summaries, revenue trends, segment analysis, and underperforming item flags, and from which filters, sub-views, and exports can be initiated.

Task 4: Initial Structural Design and Implementation Planning

4.1 UML Class Diagram

The UML Class Diagram models the full structural design of the MAS across four conceptual layers: the item hierarchy, the transaction domain, the player and user models, and the service and reporting layer. Both visual paradigm and html versions



4.2 Class Diagram: How it works

The design centres on an `ISalesItem` interface, which defines the contract - `GetPrice()`, `GetItemType()`, and `GetSalesCount()` that every purchasable item must satisfy. The abstract `MTXItem` class implements this interface and provides shared attributes (`ItemId`, `Name`, `ItemType`, `Price`, `ReleaseDate`, `SalesCount`) and shared behaviour. Six concrete subclasses extend `MTXItem`: `WeaponSkin`, `ArmourSkin`, `PetItem`, `PortalEffect`, `HideoutDecoration`, and `StashTabExpansion`, each adding type-specific attributes. `Bundle` also extends `MTXItem` and composes a list of `MTXItem` instances, representing a discounted collection.

The `Transaction` class records a single purchase event and holds associations to both an `ISalesItem` (the purchased item) and a `PlayerAccount` (the buyer), along with a captured `DemographicRecord` snapshot for historical accuracy. The immutable `DemographicRecord` decouples analytics data from the mutable `PlayerAccount`, ensuring that demographic information reported against historical transactions reflects the state at the time of purchase rather than the player's current profile.

The `SystemUser` class models internal staff with role-based access. Passwords are stored as SHA-256 hashes and never in plaintext. The `ReportEngine` class implements `IReportGenerator` and performs all analytics computation - filtering, aggregation, trend generation, and underperformance detection. `ExportService` and `TransactionService` both depend on interfaces rather than concrete implementations, satisfying the Dependency Inversion Principle.

4.3 UML to C# Class Mapping

UML Class / Interface	C# File	OOP Concept Demonstrated
«interface» <code>SalesItem</code>	<code>Models/ISalesItem.cs</code>	Interface; Dependency Inversion (SOLID)
«abstract» <code>MTXItem</code>	<code>Models/MTXItem.cs</code>	Abstraction; Encapsulation (private <code>_price</code> , <code>_salesCount</code>)
<code>WeaponSkin</code>	<code>Models/MTXItemTypes.cs</code>	Inheritance; Polymorphism via <code>ISalesItem</code>
<code>ArmourSkin</code>	<code>Models/MTXItemTypes.cs</code>	Inheritance; <code>ArmourSlot</code> enum dependency
<code>PetItem</code>	<code>Models/MTXItemTypes.cs</code>	Inheritance
<code>PortalEffect</code>	<code>Models/MTXItemTypes.cs</code>	Inheritance
<code>HideoutDecoration</code>	<code>Models/MTXItemTypes.cs</code>	Inheritance
<code>StashTabExpansion</code>	<code>Models/MTXItemTypes.cs</code>	Inheritance
<code>Bundle</code>	<code>Models/MTXItemTypes.cs</code>	Inheritance; Composition (<code>List<MTXItem></code>)
<code>Transaction</code>	<code>Models/Transaction.cs</code>	Encapsulation; Association; Immutability
<code>DemographicRecord</code>	<code>Models/Transaction.cs</code>	Immutable Value Object; Decoupling pattern
<code>PlayerAccount</code>	<code>Models/PlayerAccount.cs</code>	Encapsulation; <code>HashSet</code> for purchase history
<code>SystemUser</code>	<code>Models/SystemUser.cs</code>	Encapsulation (SHA-256 hash); Principle of Least Privilege
<code>FilterSet</code> / <code>DateRange</code> / <code>Report</code>	<code>Models/ReportModels.cs</code>	Value Objects; Data Transfer Object pattern
«interface» <code>IReportGenerator</code>	<code>Services/IReportGenerator.cs</code>	Interface Segregation; Dependency Inversion
<code>ReportEngine</code>	<code>Services/ReportEngine.cs</code>	Single Responsibility; High Cohesion
<code>TransactionService</code>	<code>Services/TransactionService.cs</code>	Service Layer; Low Coupling via repository interfaces
<code>AuthService</code>	<code>Services/AuthService.cs</code>	Encapsulation; Session management
<code>ExportService</code>	<code>Services/ExportService.cs</code>	Single Responsibility; Strategy-like dispatch

4.4 OOP Principles Applied

4.4.1 Abstraction

Abstraction is realised through the `ISalesItem` interface and the abstract `MTXItem` base class. The interface exposes only three operations - `GetPrice()`, `GetItemType()`, and `GetSalesCount()` - to consuming classes such as `Transaction` and `ReportEngine`. Neither the `Transaction` class nor the reporting logic needs to know whether the item it is processing is a `WeaponSkin` or a `PetItem`; it only needs to know that it is a `SalesItem`. This abstraction reduces coupling and enables the system to accommodate new item types without modifying existing service code.

4.4.2 Encapsulation

Encapsulation is consistently applied throughout the model layer. In `MTXItem`, the `_price` and `_salesCount` fields are private; external code can only read price via `GetPrice()` and can only increment the sales count via `IncrementSales()`, preventing arbitrary modification. In `SystemUser`, the password is immediately hashed on construction via SHA-256 and the plaintext is never stored; the `_passwordHash` field is private, and password verification is only possible through the `Authenticate()` method. In `PlayerAccount`, the purchase history is maintained as a private `HashSet`, and only an `IReadOnlyCollection` is exposed externally, preventing external mutation.

4.4.3 Inheritance

All six concrete item classes - `WeaponSkin`, `ArmourSkin`, `PetItem`, `PortalEffect`, `HideoutDecoration`, and `StashTabExpansion` - inherit from the abstract `MTXItem` class. Each subclass calls the base constructor to initialise shared fields and adds only the attributes specific to its type (e.g. `WeaponSkin` adds `WeaponClass` and `EffectTier`; `ArmourSkin` adds `SlotType` and `AnimSet`). `Bundle` also inherits from `MTXItem` and extends it with composition, maintaining an internal list of `MTXItem` instances that it contains.

4.4.4 Polymorphism

Polymorphism is exercised through the `ISalesItem` interface reference used in the `Transaction` class. The `_item` field is typed as `ISalesItem`, not as `MTXItem` or any concrete subclass. The `ReportEngine` operates on `IEnumerable<Transaction>` and accesses item information exclusively through the `GetItemType()` and `GetPrice()` methods defined on `ISalesItem`. This means a new item type can be introduced by creating a new `MTXItem` subclass without modifying `Transaction`, `ReportEngine`, or `ExportService`. The abstract `GetDescription()` method in `MTXItem` is overridden by each subclass, providing polymorphic behaviour in UI display contexts.

4.5 Anticipated Design Weaknesses and Critical Evaluation

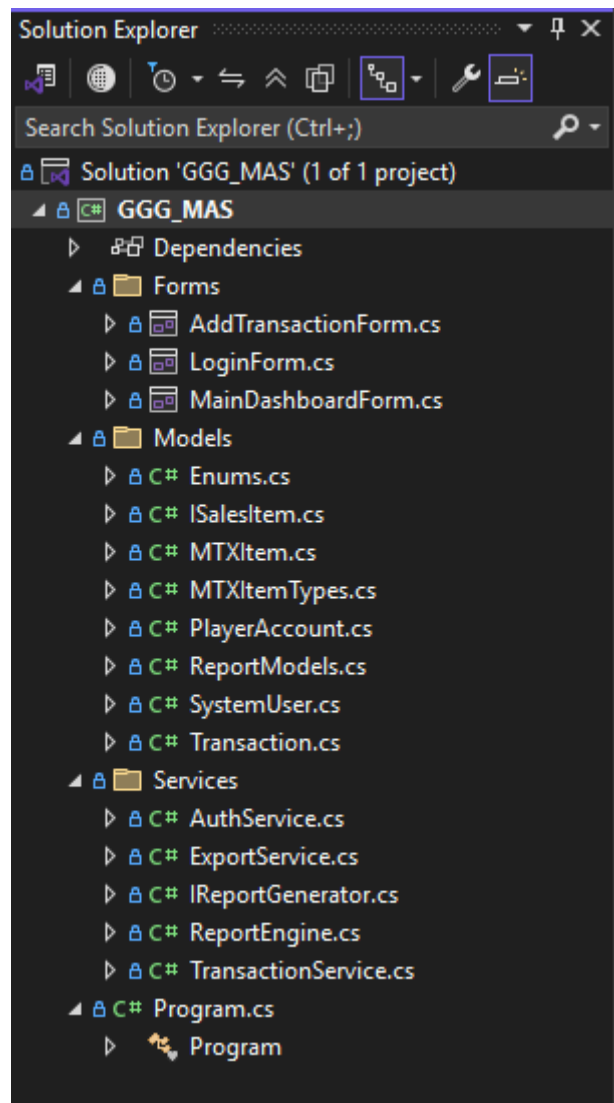
A critical evaluation of the current design identifies the following limitations, which should be addressed in subsequent development phases:

Design Weakness	Impact	Recommended Mitigation
In-memory repositories with no persistence	All transaction and catalogue data is lost when the application closes. Not viable for production use where GGG processes millions of transactions.	Replace <code>InMemoryTransactionRepository</code> and <code>InMemoryItemRepository</code> with SQL-backed implementations. The repository interface pattern means existing service code requires no changes.
WinForms UI is Windows-only	GGG's development and analytics teams may use macOS or Linux. The UI layer cannot be deployed cross-platform.	Migrate the UI to a web-based framework (e.g. ASP.NET Core Razor Pages or a JavaScript SPA with a REST API backend) to enable platform-independent access.
No real-time event streaming	NFR01 requires purchase data within 2 seconds of transaction completion. The current design uses a synchronous method call, which would need to be replaced with an event-driven ingestion pipeline in production.	Introduce a message queue (e.g. RabbitMQ or Azure Service Bus) between the game server and MAS. <code>TransactionService</code> becomes a consumer rather than a direct call target.
Single-user session model	<code>AuthService</code> holds one <code>CurrentUser</code> at a time, making it incompatible with NFR04's requirement for 50 concurrent users.	Refactor <code>AuthService</code> to a session-token model where each login creates a scoped session object, supporting concurrent authenticated users independently.
No data anonymisation layer	NFR03 and SC-09 require player data to be anonymised before surfacing to non-Analytics roles. Currently, all roles can access <code>DemographicRecord.Region</code> and <code>PlayerAccount</code> fields without filtering.	Introduce a data access layer that applies role-based field masking before returning <code>PlayerAccount</code> or <code>DemographicRecord</code> data to non-Analyst users.

4.6 C# Implementation Overview

The C# implementation is structured across three namespaces: GGG_MAS.Models (domain objects and enumerations), GGG_MAS.Services (business logic and data access), and GGG_MAS.Forms (WinForms user interface). The implementation consists of 17 source files and a single solution file. The application bootstraps via Program.cs, which performs manual dependency composition - seeding repositories with 250 sample transactions across 20 catalogue items and 10 player accounts - and launches a login screen before proceeding to the main dashboard.

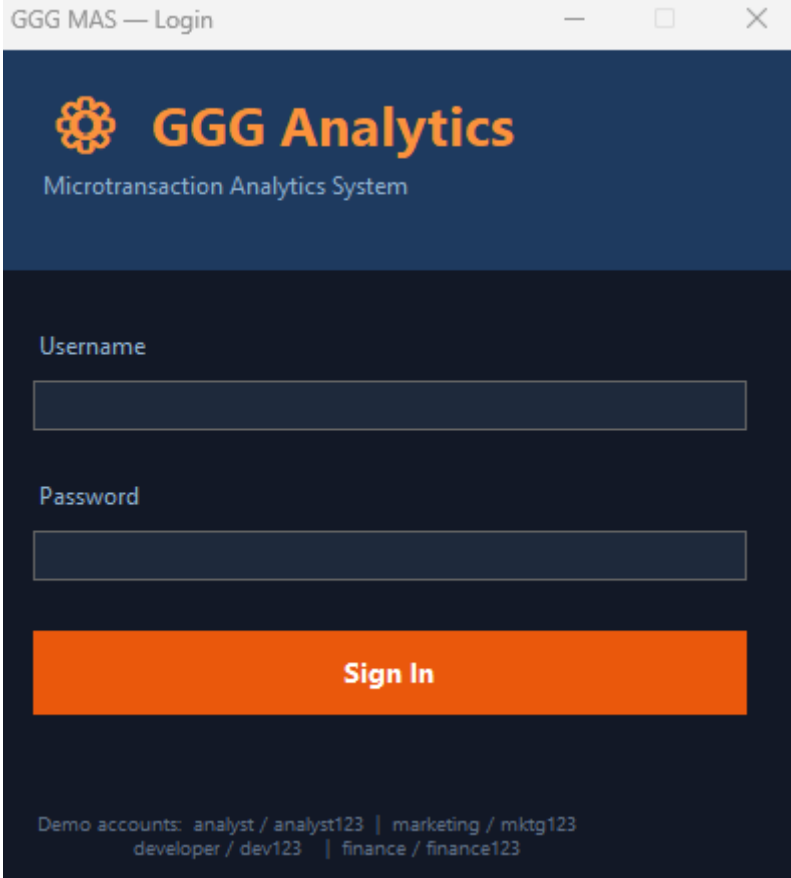
Core functionality implemented and demonstrable includes: recording new purchases via the Add Transaction form; generating and viewing analytics reports with live filter controls; identifying and displaying underperforming items; exporting reports in CSV, PDF text, and JSON format; and role-based access enforcement preventing unauthorised users from accessing export and configuration capabilities.



4.6 .1 Microtransaction Analytics System demo

Screenshots of the GUI MAS

Login interface



GGG MAS — Login

 **GGG Analytics**
Microtransaction Analytics System

Username

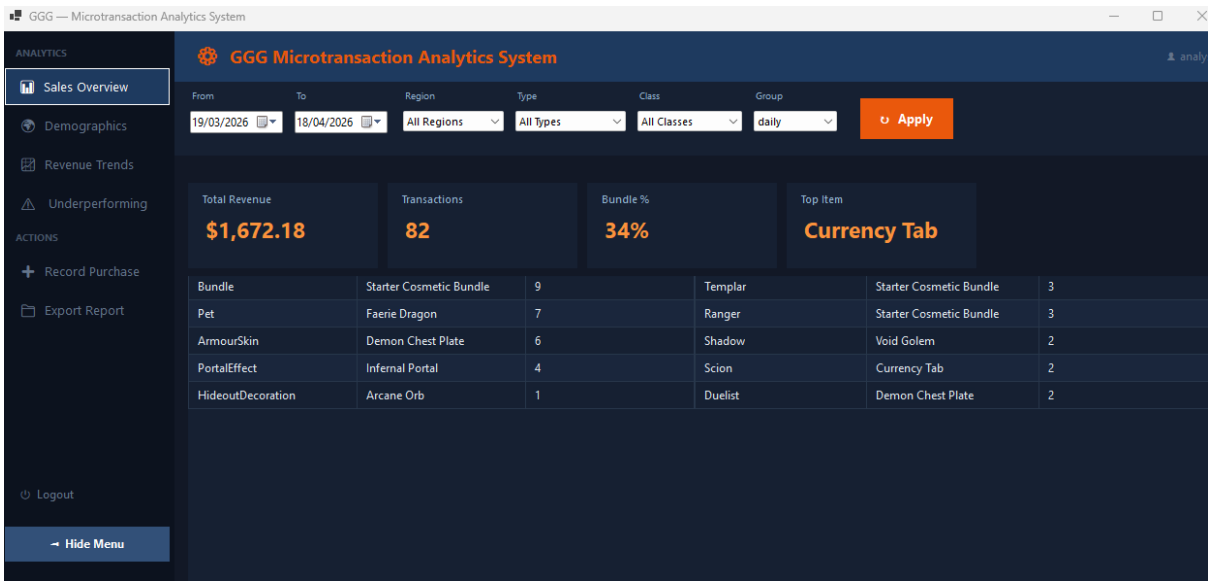
Password

Sign In

Demo accounts: analyst / analyst123 | marketing / mktg123
developer / dev123 | finance / finance123

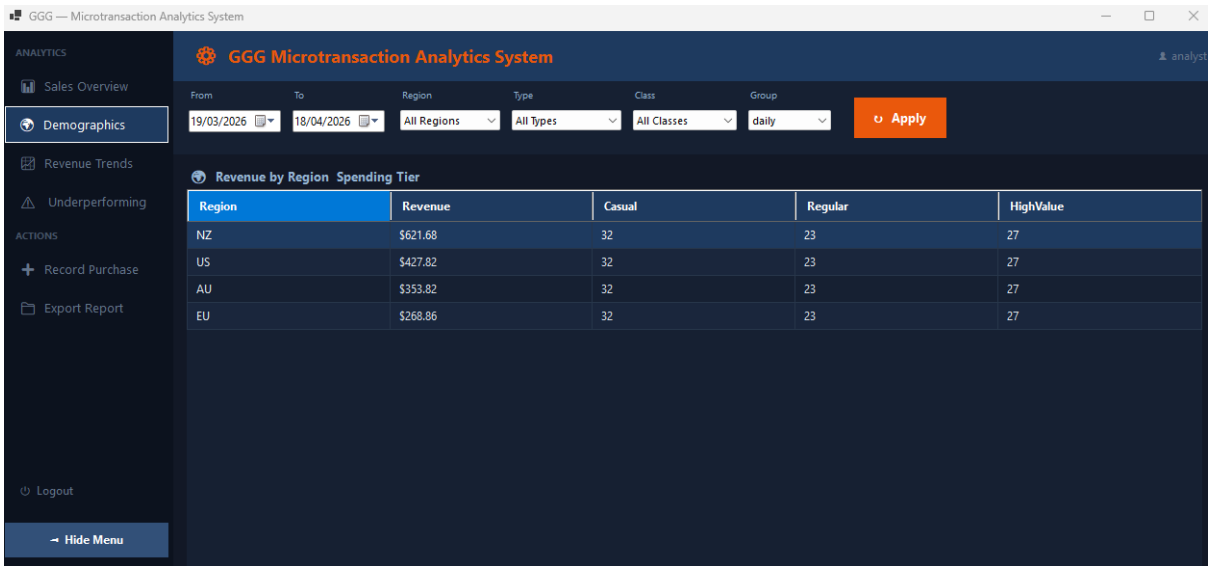
The LoginForm is the first screen displayed when the application launches and requires the user to enter a valid username and password before accessing the dashboard. It delegates authentication to the AuthService class, which checks the provided credentials against stored password hashes, and on success establishes a session by storing the current user and their role for the lifetime of the application. Failed attempts display a generic error message without revealing which field was incorrect, the password field is cleared automatically, and the form remains open so the user can retry.

Sales Overview Shows cosmetic sales data



The Sales Overview is the default view shown on dashboard load and presents four KPI stat cards at the top displaying total revenue, transaction count, bundle purchase percentage, and the best-selling item name, all calculated from the current report. Below the cards, two side by side data grids show the top-selling item per category and the top-selling item per character class, both sorted descending by sales count and responsive to the active filter bar settings. The entire view is regenerated on demand whenever the user adjusts the date range, region, item type, or character class filters and clicks Apply.

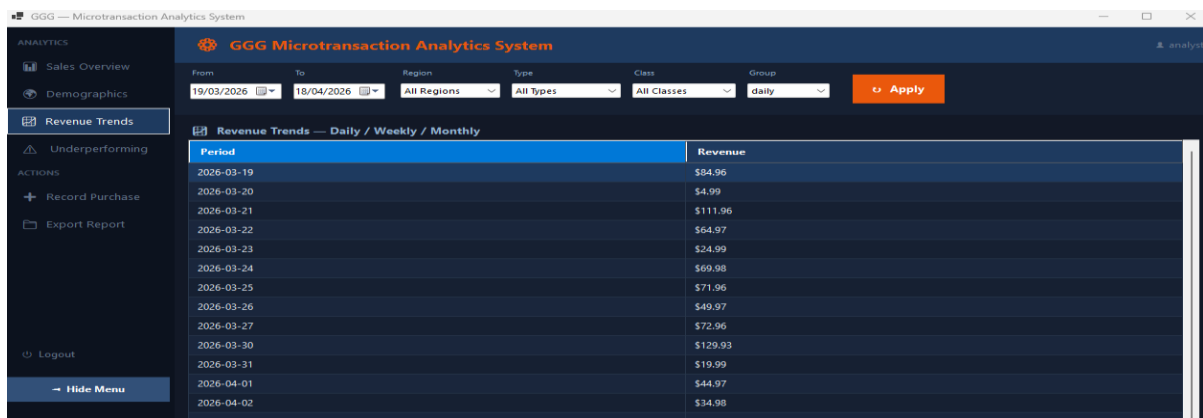
Demographic Shows how each region sales are



The Demographics view presents a single data grid that cross-references geographic region against player spending tier, showing each region's total revenue alongside a breakdown of how many Casual, Regular, and High-Value players contributed to that figure. The data is sourced directly from the report's RevenueByRegion and TierDistribution dictionaries and rows are ordered by revenue descending so the highest-contributing markets appear first. This view allows Marketing to identify which regions host the most high-value players and target promotional campaigns accordingly.

Revenue Trends

Shows daily, weekly and monthly sales



GGG Microtransaction Analytics System

From: 19/03/2026 To: 18/04/2026 Region: All Regions Type: All Types Class: All Classes Group: daily Apply

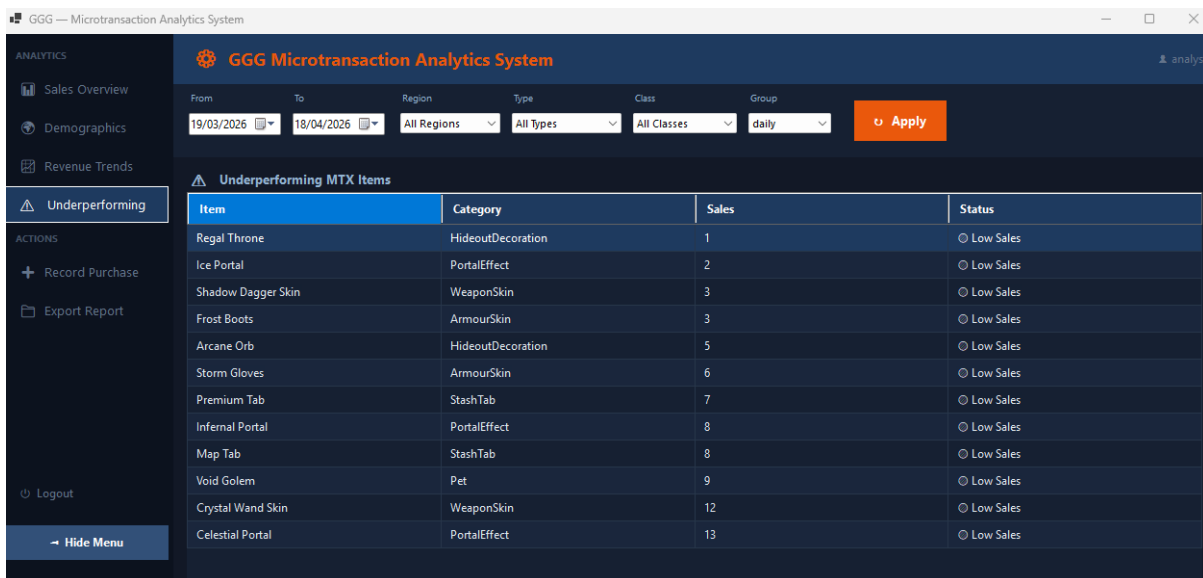
Revenue Trends — Daily / Weekly / Monthly

Period	Revenue
2026-03-19	\$84.96
2026-03-20	\$4.99
2026-03-21	\$111.96
2026-03-22	\$64.97
2026-03-23	\$24.99
2026-03-24	\$69.98
2026-03-25	\$71.96
2026-03-26	\$49.97
2026-03-27	\$72.96
2026-03-30	\$129.93
2026-03-31	\$19.99
2026-04-01	\$44.97
2026-04-02	\$34.98
2026-04-03	\$16.99

The Revenue Trends view displays a single data grid with two columns, Period and Revenue, showing how total microtransaction revenue is distributed across time at a granularity selected by the user from daily, weekly, or monthly groupings. The grouping is controlled by the Group filter in the filter bar and is passed into the ReportEngine when the report is generated, with rows sorted chronologically so the table reads as a time series. This gives Finance a clear picture of revenue movement over any selected window and supports forecasting and budget planning without requiring manual data extraction.

Underperforming

Shows which cosmetic items are not doing well



GGG Microtransaction Analytics System

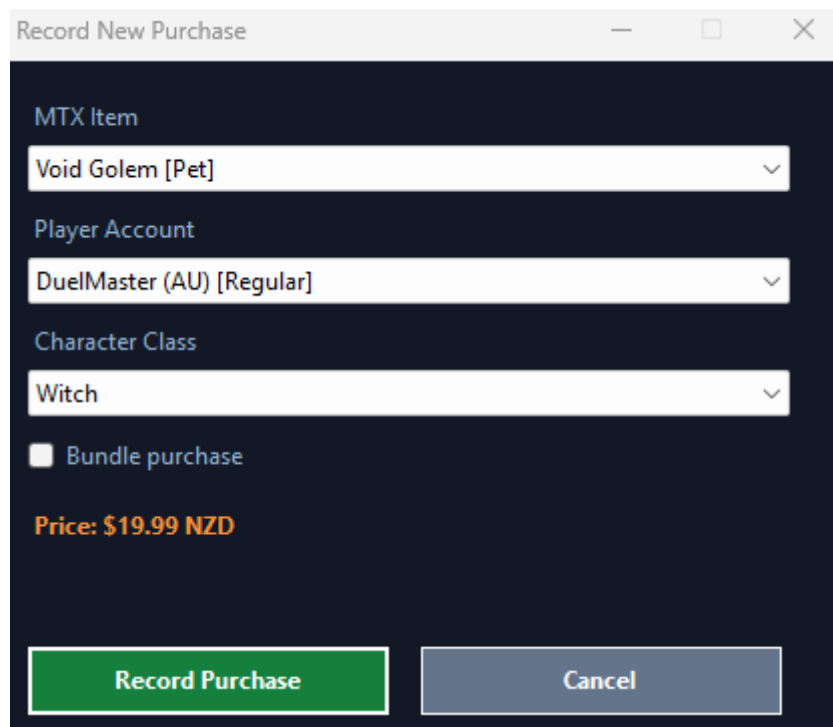
From: 19/03/2026 To: 18/04/2026 Region: All Regions Type: All Types Class: All Classes Group: daily Apply

Underperforming MTX Items

Item	Category	Sales	Status
Regal Throne	HideoutDecoration	1	Low Sales
Ice Portal	PortalEffect	2	Low Sales
Shadow Dagger Skin	WeaponSkin	3	Low Sales
Frost Boots	ArmourSkin	3	Low Sales
Arcane Orb	HideoutDecoration	5	Low Sales
Storm Gloves	ArmourSkin	6	Low Sales
Premium Tab	StashTab	7	Low Sales
Infernal Portal	PortalEffect	8	Low Sales
Map Tab	StashTab	8	Low Sales
Void Golem	Pet	9	Low Sales
Crystal Wand Skin	WeaponSkin	12	Low Sales
Celestial Portal	PortalEffect	13	Low Sales

The Underperforming Items view displays a data grid listing every catalogue item whose sales count falls below the configured threshold, with columns for item name, category, sales count, and a computed status field that labels items as either No Sales or Low Sales. Rows where the sales count is zero are additionally coloured red to make them immediately distinguishable from items that are merely underperforming rather than completely inactive. This view allows Game Development and Marketing to quickly identify which cosmetics need to be promoted, discounted, or retired based on actual sales data rather than estimation.

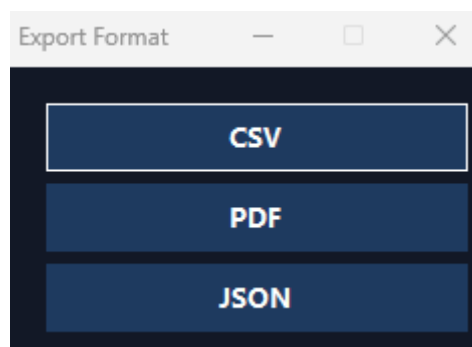
Records any interesting or outlier purchases

A modal dialog box titled "Record New Purchase" with a dark blue background. It contains three dropdown menus: "MTX Item" with "Void Golem [Pet]" selected, "Player Account" with "DuelMaster (AU) [Regular]" selected, and "Character Class" with "Witch" selected. Below these is a checkbox labeled "Bundle purchase" which is unchecked. A price label "Price: \$19.99 NZD" is displayed in orange text. At the bottom are two buttons: a green "Record Purchase" button and a grey "Cancel" button.

The Record New Purchase dialog is a modal form that allows authorised users to manually log a new microtransaction, requiring them to select an MTX item, a player account, and a character class, with an optional checkbox to flag the purchase as a bundle. A live price label updates automatically when an item is selected, and the form validates that all required fields are filled before delegating the actual transaction logic to the TransactionService. Once submitted successfully, the dashboard refreshes immediately so all views and KPI cards reflect the newly recorded transaction

Export format

Allows different forms Of export

A modal dialog box titled "Export Format" with a dark blue background. It contains three buttons stacked vertically: "CSV", "PDF", and "JSON".

The Export Report feature is only visible to users whose role has export permission, and when triggered it opens a small format-picker dialog presenting three buttons for CSV, PDF, and JSON. After a format is selected, a standard save dialog opens with the appropriate file extension and a timestamped default filename, and the report is written to disk by the ReportEngine and its underlying ExportService. Any file write errors are caught and displayed to the user in a clear error message without exposing internal application details.

4.7 GitHub Repository and Compilation Instructions

GitHub Repository URL:

<https://github.com/Suemon-Kwok/ENSE706-Data-Process-and-Software-Modelling-2026-S1---Microtransaction-Analytics-System---14883335>

Prerequisites

- .NET 8.0 SDK or later
- Visual Studio 2022 (Community or higher) with Windows Forms workload, OR .NET CLI
- Windows operating system (WinForms requirement)

Build and Run Steps

1. Clone the repository: `git clone [repository URL]`
2. Open GGG_MAS.sln in Visual Studio 2022
3. Build the solution: Build menu > Build Solution (Ctrl+Shift+B)
4. Run the application: Debug menu > Start Debugging (F5)
5. At the login screen, use one of the demo credentials listed below

Username	Password	Role	Capabilities
analyst	analyst123	Data Analyst	Full system access
marketing	mktg123	Marketing Manager	Reports + Export
finance	finance123	Finance Officer	Revenue reports + Export
developer	dev123	Game Developer	Item data + Underperformance

References

Grinding Gear Games. (2024). *Path of Exile – Official site*. <https://www.pathofexile.com>

Grinding Gear Games. (2026). *Grinding Gear Games*. <https://www.grindinggear.com/>

New Zealand Companies Office. (2006). *Grinding Gear Games Limited (1887410)*. New Zealand Companies Register. <https://app.companiesoffice.govt.nz/companies/app/ui/pages/companies/1887410>

PitchBook. (2025). *Grinding Gear Games company profile*. <https://pitchbook.com/profiles/company/228913-48#overview>

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.

ZoomInfo. (2025). *Grinding Gear Games*. <https://www.zoominfo.com/pic/grinding-gear-games/346696694>